7

FULL-STACK RAG · STEP BY STEP

Steps to Build a RAG AI Agent

PDF in, grounded answers out. FastAPI + FAISS + React.



Chandra Sekhar
AI & Tech

Save

THE GOAL

What You'll Build

00

A support bot that answers **only from your documents**, not from the model's imagination.

Reads PDFs
Your policies, docs or FAQs become the knowledge base.

Stores embeddings
FAISS keeps meaning-vectors searchable on disk.

Retrieves per question
Top matching chunks, every single query.

Ships as a product
FastAPI backend plus a React chat widget.

> **Not a demo. The same skeleton runs real insurance, HR and SaaS support bots.**



Chandra Sekhar
AI & Tech

Follow me

THE IDEA

RAG in One Sentence

01

Search your documents first, then generate the answer. The model stops guessing and starts quoting your data.

1. User asks a question
2. Search your documents for matches
3. Send only those chunks to the LLM
4. Answer grounded in real text

GREAT FIT FOR

- Insurance and legal docs
- HR policies
- SaaS documentation
- Customer support

Retrieval quality decides answer quality.



Chandra Sekhar
AI & Tech

Follow me

STACK

The Tech Stack

02

Small, boring, proven tools. Nothing here needs a GPU.

Backend

FastAPI, PyPDF, tiktoken, FAISS, OpenAI embeddings + Responses API

FRONTEND

React + Vite, one simple chat widget

PyPDF

Pulls raw text out of the PDF.

tiktoken

Token-accurate chunking with overlap.

FAISS

Fast local vector search, no server.

> **Pick tools you can debug at 2 AM.**



Chandra Sekhar
AI & Tech

Follow me

SETUP

Folder Structure

03

One clean layout. Reuse it for healthcare, finance, education or SaaS support.

project layout

```
insurance-rag-bot/
  backend/
    main.py
    rag/
      pdf_to_text.py
      chunking.py
      embed_store.py
      rag_answer.py
    data/knowledge.pdf
  frontend/
    src/App.jsx
    src/ChatWidget.jsx
```

One file per pipeline stage. Easy to swap later.

STEP 01

Read the PDF

01

Extract text page by page, then strip blank lines and stray carriage returns.

rag/pdf_to_text.py

```
from pypdf import PdfReader

def pdf_to_text(path: str) → str:
    reader = PdfReader(path)
    pages = [p.extract_text() or ""
             for p in reader.pages]
    text = "\n".join(pages)
    lines = [l.strip() for l in
             text.split("\n") if l.strip()]
    return "\n".join(lines)
```

⚠ **Scanned PDFs**

Image-only files need OCR first. Text PDFs work directly.

Garbage text in means garbage answers out.



Chandra Sekhar
AI & Tech

Follow me

STEP 02

Chunk It Right

02

The most important step. Split into small **overlapping** token windows so search stays reliable.

rag/chunking.py

```python
import tiktoken

def chunk_text(text, size=450, overlap=80):
    enc = tiktoken.get_encoding("cl100k_base")
    toks = enc.encode(text)
    out, start = [], 0
    while start < len(toks):
        end = start + size
        out.append(enc.decode(toks[start:end]))
        start = end - overlap
    return out
```

TOO LARGE

Messy, unfocused answers

TOO SMALL

Context gets lost

Start at 450 tokens with 80 overlap, then tune.



Chandra Sekhar
AI & Tech

Follow me

CHUNKING

Four Chunking Strategies

03

Token chunking is the default. Upgrade only when your documents demand it.

Token chunking
Best general default. Fast and predictable.

Section-based
Split on headings for structured documents.

Semantic
Highest quality, but more complex to run.

Hybrid
Combine strategies, for example section then token.

Rule of thumb
FAQ or policy PDF? Section-based. Long prose? Token with overlap.

Measure retrieval hits before adding complexity.



Chandra Sekhar
AI & Tech

Follow me

STEP 03

Embed and Store

04

Embeddings turn text into vectors that capture **meaning**. FAISS stores them for fast search.

rag/embed_store.py

```
def embed_texts(texts):
    r = client.embeddings.create(
        model="text-embedding-3-small",
        input=texts)
    arr = np.array([d.embedding for d in r.data],
                   dtype="float32")
    faiss.normalize_L2(arr)
    return arr

def build_index(chunks, path):
    vecs = embed_texts(chunks)
    index = faiss.IndexFlatIP(vecs.shape[1])
    index.add(vecs)
    faiss.write_index(index, path)
```

Normalize vectors so inner product equals cosine.



Chandra Sekhar
AI & Tech

Follow me

STEP 04

Retrieve the Top Chunks 05

Embed the question the same way, then ask FAISS for the **top-k** closest chunks.

rag/rag_answer.py

```
def embed_query(q):
    r = client.embeddings.create(
        model=EMBED_MODEL, input=q)
    v = np.array([r.data[0].embedding],
                 dtype="float32")
    faiss.normalize_L2(v)
    return v

def retrieve(q, index, chunks, k=4):
    _, ids = index.search(embed_query(q), k)
    return [chunks[i] for i in ids[0] if i ≠ -1]
```

Tip

Normalize the query too, or scores will not match the index.

k=4 is a solid start. Raise it if answers miss context.

STEP 05

Grounded Answers

06

This is the safety step. The model may only use the retrieved context.

rag/rag_answer.py

```
def generate_answer(question, hits):
    context = "\n\n".join(hits)
    r = client.responses.create(
        model=CHAT_MODEL,
        instructions=(
            "You are a customer care assistant. "
            "Use only the provided context."),
        input=f"Context:\n{context}\n\n"
              f"Question:\n{question}")
    return r.output_text
```

Context in the prompt, rules in the instructions. That combo cuts hallucinations.

Tell it to say "I don't know" when context is empty.



Chandra Sekhar
AI & Tech

Follow me

STEP 06

Wire Up FastAPI

07

Two endpoints: **/ingest** builds the index once, **/chat** answers questions.

main.py

```
@app.post("/ingest")
def ingest():
    text = pdf_to_text(PDF_PATH)
    chunks = chunk_text(text)
    build_and_save_index(
        chunks, INDEX_PATH, META_PATH)
    return {"status": "ok", "chunks": len(chunks)}

@app.post("/chat")
def chat(p: ChatIn):
    hits = retrieve(p.message, index, chunks)
    return {"answer": generate_answer(p.message, hits)}
```

Load the saved index lazily if the server restarts.

STEP 07

React Chat Widget

08

A tiny component: keep messages in state, POST to **/chat**, append the reply.

ChatWidget.jsx

```
async function send() {
  const msg = text.trim();
  if (!msg) return;
  setMsgs(m => [...m, {role:"user", text:msg}]);
  setText("");
  const res = await fetch("/chat", {
    method: "POST",
    headers: {"Content-Type":"application/json"},
    body: JSON.stringify({ message: msg }),
  });
  const data = await res.json();
  setMsgs(m => [...m, {role:"bot", text:data.answer}]);
}
```

Add loading and error states before you ship.



Chandra Sekhar
AI & Tech

Follow me

RUN IT

Launch the Project

09

Backend first, index once, then the frontend.

1. pip install -r requirements.txt
2. python rag/make_sample_pdf.py
3. uvicorn main:app --reload --port 8000
4. curl -X POST localhost:8000/ingest
5. cd frontend, npm install, npm run dev

🔑 **Protect your key**

Put OPENAI_API_KEY in backend/.env and never commit it.

Requirements: fastapi, uvicorn, pypdf, tiktoken, faiss-cpu, numpy, openai.



Chandra Sekhar
AI & Tech

Follow me

YOUR RAG AGENT IS READY

Ship it.
Then make it better.

PDF to answers in 7 steps. Share this with someone building AI that actually works.

Repost Save Comment

Follow for more AI